JETBRAINS | Kotlin

Build Systems

kotlinglang.org/education



What? Why?

Build system – Software that automates the process of getting some kind of an artifact (executable, library) from the source code. Build systems can be used for:

Configuring your build once and using it forever ~~(copy-paste into new projects)~~

-
- Unifying builds and reusing logic in various projects
- Dependencies management*
- Testing and verification
- Incremental builds*

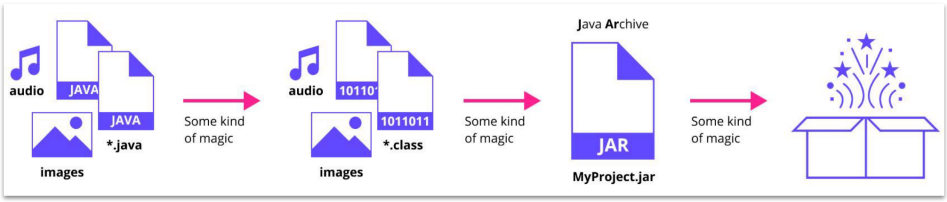








How?


audio
JAVA
JAVA
*.java
images
Some kind of magic
audio
1011011
*.class
images
Some kind of magic
Java Archive
JAR
MyProject.jar
Some kind of magic

Maven



pom.xml

Project **O**bject **M**odel

Declarative: You define the configuration without specifying how to achieve it.

Convention: You describe what you need with specific rules.

Lifecycle: It can support everything from compilation to tests and so on.

Plugins allow you to do the unconventional heavy-lifting.

Coordinates are located in pom.xml: *groupId*, *artifactId*, *version*.

Repositories: You can load (and cache) the dependencies on demand.

Learn more: *search.maven.org* (Maven Central)

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
maven.apache.org/xsd/maven-4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>

<groupId>com.mycompany.app</groupId>
<artifactId>my-app</artifactId>
<version>1.0-SNAPSHOT</version>

<properties>
        <maven.compiler.source>1.7</maven.compiler.source>
        <maven.compiler.target>1.7</maven.compiler.target>
</properties>

        <dependencies>
                <dependency>
                        <groupId>junit</groupId>
                        <artifactId>junit</artifactId>
                        <version>4.12</version>
                        <scope>test</scope>
                </dependency>
        </dependencies>
</project>
```

Gradle



build.gradle

settings.gradle

DSL: It uses Kotlin or Groovy instead of XML.

Tasks: You can define actions which might depend on each other and be quite complex.

Plugins provide unconventional predefined tasks to do the heavy-lifting.

Modules have independent compilation units. Each unit is built into a separate JAR (or some other kind of artifact).

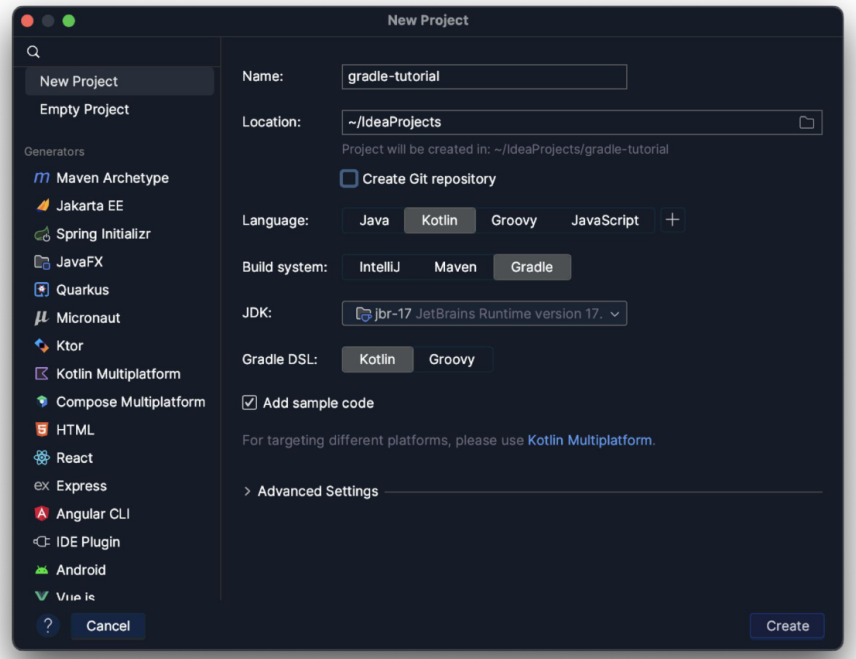
Repositories: You can reuse Maven repositories.

Dependency Management: You can easily declare and resolve dependencies.

Language Agnostic: Gradle can be used for Kotlin, Java, Scala, C++, JS, and COBOL.

Learn more: *docs.gradle.org*

New Gradle Project in IntelliJ IDEA



Java bytecode

```
plugins {
    kotlin("jvm") version "1.8.0"
    application
}


group = "org.constructor.jetbrains.kotlin"
version = "1.0-SNAPSHOT"


repositories {
    mavenCentral()
}


dependencies {
    testImplementation(kotlin("test"))
}


tasks.test {
    useJUnitPlatform()
}


kotlin {
    jvmToolchain(8)
}


application {
    mainClass.set("MainKt")
}


Failed to calculate the value

of task ':compileJava' property

'javaCompiler'. Unable to download

toolchain matching these requirements:

languageVersion=8, vendor=any,

implementation=vendor-specific

Unable to download toolchain. This

might indicate that the combination

(version, architecture, release/early

access, …) for the requested

JDK is not available. Could not read

'https:^/api.adoptopenjdk.net/v3/binary/latest/8/ga/mac/aarch64/jdk/h'

as it does not exist.
```

Java bytecode

I use JetBrains Runtime 17.0.5, so my JDK version is 17, and to fix the problem I do:

```
kotlin {
    jvmToolchain(17)
}
```

But usually instead of `jvmToolchain` you will see:

```
tasks.withType<KotlinCompile> {
    kotlinOptions.jvmTarget = "1.8"
}
```

Build script

Gradle tries hard to look like a declarative build configuration while using a procedural language (Groovy or Kotlin).

The first thing we see in `build.gradle.kts` is the plugins `{ ^^. }` block, which is a method of:

```
class
org.gradle.kotlin.dsl.KotlinBuildScript
:> abstract class ProjectDelegate
:> interface Project : Comparable,
ExtensionAware, PluginAware
```

Project

The `Project` interface is the main API you use to interact with Gradle from your build file. From a `Project`, you have programmatic access to all of Gradle's features. There is a one-to-one relationship between `Projects` and `build.gradle` files.

Lifecycle:

1.Assemble a `Project` object for each project.

2.Create an `org.gradle.api.initialization.Settings` instance for the build. 3.Evaluate the `settings.gradle` script, if present, using the `Settings` object. 4.Use the `Settings` object to create the hierarchy of `Project` instances. 5.Evaluate each `Project` by executing its `build.gradle` file.

Project: Settings

If we take the following steps:

Project → New → Module

A new folder (which is a new module) appears. It has its own `build.gradle.kts`, but no

`settings.gradle.kts`, while a new line is added to the settings file in the root:

```
rootProject.name = "gradle-tutorial"
```

```
include("module")
```

Sidenote: For a single-project build, a settings file is optional.

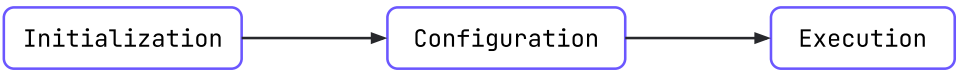
Project: Settings

A small addition to settings.gradle.kts:

```
rootProject.name = "gradle-tutorial"
include("module")
println("Initialization phase.")
```

```
Initialization phase.
> Task :prepareKotlinBuildScriptModel
UP-TO-DATE

BUILD SUCCESSFUL in 3s
```



Project: Tasks

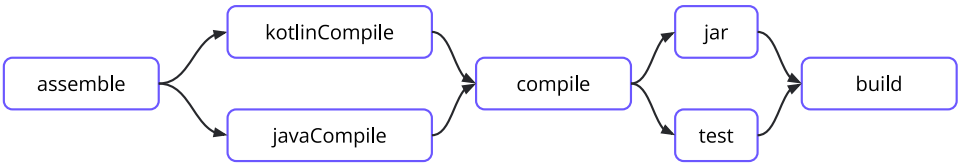
A project is essentially a collection of `Task` objects. Each task performs some basic piece of work, such as compiling classes, running unit tests, or zipping up JAR files.

There are:

- Default tasks;
- Tasks from plugins;
- Custom tasks, defined by the user right in the build configuration.

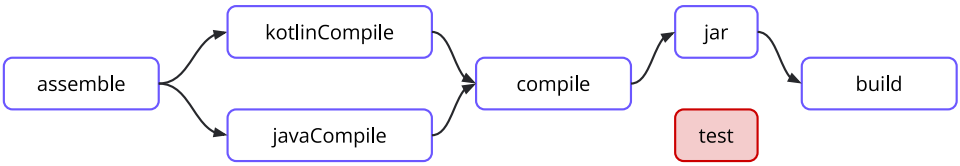
Tasks can depend on each other. Gradle builds an acyclic directed graph of tasks.

Tasks




```
./gradlew build
```

Tasks




```
./gradlew build -x test
```

Build without running tests.

Project: Tasks

The configuration language is the Kotlin (or Groovy) DSL, so anything is possible. You can add behavior to already existing tasks.

```
fun Task.printName() = println("Hi! My name is $
{this.name}")

val task3 = tasks.register("thirdTask") {
    doFirst { printName() }
}

tasks.filter { task → task.group?.let { it ≠
"useless" } ?: false }
     .forEach {
         it.dependsOn(task3)
     }

tasks.test {
    useJUnitPlatform()
}
```

Project: Tasks

Tasks have inputs and outputs. Gradle caches the results of tasks. If on a new run inputs hash has not changed, Gradle uses the previous result there: UP-TO-DATE. If hash changed, but it is present in cache, then: FROM-CACHE. Otherwise, it still tries to reuse previous results.

Project: Tasks

```
tasks.register("targetTask") {
    group = "useless"
    dependsOn(tasks.named("dependencyTask"))
    println("${this.name}, configuration")
    doFirst {
        println("${this.name}, first in execution")
    }
}

tasks.register("dependencyTask") {
    println("${this.name}, configuration")
    doFirst {
        println("${this.name}, first in execution")
    }
    doLast {
        println("${this.name}, last in execution")
    }
}
```

```
./gradlew :targetTask

Executing 'targetTask'...

> Configure project :
targetTask, configuration
dependencyTask, configuration

> Task :dependencyTask
dependencyTask, first in execution
dependencyTask, last in execution

> Task :targetTask
targetTask, first in execution
```

Plugins

Most of Gradle's useful features are added with plugins in the `plugins` block of a configuration script.

Plugins add new tasks (e.g. `JavaCompile`), domain objects (e.g. `SourceSet`), and conventions (e.g. locating Java source at `src/main/java`), and they can also extend core objects and objects from other plugins.

There are binary plugins and script plugins. Binary plugins are usually an external plugin jar. Script plugins are typically used within a build.

Plugins

Gradle provides the core plugins (e.g. `application`, `java`) as part of its distribution.

Applying a community plugin:

```
plugins {
    application // core plugin
    kotlin("jvm") version "1.8.0" //
id("org.jetbrains.kotlin.jvm") version "1.8.0"
    id("io.gitlab.arturbosch.detekt") version
"1.21.0" apply false // later apply it in a subproject
}
```

There are also plugins that are built from `buildSrc` and then applied by `id` as though they were community plugins.

Custom plugin repositories can be added via the `pluginManagement { … }` block in `settings.gradle.kts`.

Plugins

```
// our build.gradle.kts
plugins {
    kotlin("jvm") version "1.8.0"
    application
}

kotlin { // provided by kotlin("jvm") community
plugin
    jvmToolchain(8)
}

application { // provided by application core plugin
    mainClass.set("MainKt")
}
```

With this snippet, we’ve applied two plugins to our project.

Plugins

```
// our build.gradle.kts
plugins {
    kotlin("jvm") apply false
    kotlin("plugin.serialization") apply false
}

allProjects {
    apply {
        plugin("kotlin")
    }
}
```

Here we've applied only the Kotlin plugin to all modules in the project eagerly.

Plugins

```
// our build.gradle.kts
plugins {
    kotlin("jvm") apply false
    kotlin("plugin.serialization") apply false
}

…

val ignored = listOf("common")

configure(subprojects.filter { it.name !in ignored })
{
    apply {
        plugin("kotlinx-serialization")
    }
}
```

We applied the kotlinx-serialization plugin to all modules except common.

Dependency management

Gradle will look for declared dependencies in repositories, which are local directories or remote repositories. This process is called dependency resolution.

Once the dependencies are resolved, the resolution mechanism stores the underlying files of a dependency in a local cache (the local Maven repository).

Gradle handles dependency resolution in the event that there are conflicting transitive dependencies. This behavior can also be customized.

Dependency management: Repositories

```
repositories {
    mavenCentral()
    maven {
        url = uri("https://
your.company.com/maven")
            credentials {
                username = USER_NAME
                password = PASSWORD
            }
        }
    flatDir {
        dirs("libraries")
    }
}
```

Dependency management: Dependencies

```
val ktor_version: String by project

dependencies {

// the string notation, e.g. group:name:version
implementation("io.ktor:ktor-server-core-jvm:
$ktor_version")

// map notation + api (internals are accessible)

api("io.ktor", "ktor-server-netty-jvm", ktor_version)

// dependency on another project

implementation(project(":neighborProject"))

// putting all jars from 'libs' onto compile
classpath

implementation(fileTree("libs"))

// test dependency
```

Dependency management: Dependencies

```
// settings.gradle.kts

val utilitiesRepo = "https:^/github.com/JetBrains-Research/plugin-utilities.git"

val utilitiesProjectName = "org.jetbrains.research.pluginUtilities"

sourceControl {

    gitRepository(URI.create(utilitiesRepo)) {

        producesModule("$utilitiesProjectName:plugin-utilities-core")

    }

}

// build.gradle.kts

val utilitiesProjectName = "org.jetbrains.research.pluginUtilities"

dependencies {

    implementation("$utilitiesProjectName:plugin-utilities-core") {

        version {

            branch = main
```

Gradle properties

Properties are used to configure the behavior of Gradle itself and specific projects. From highest to lowest precedence:

- Command-line flags, such as `--build-cache`
- Properties stored in a local `gradle.properties` file.
- Properties stored in the `~/.gradle/gradle.properties` file.
- Gradle properties, such as `org.gradle.caching=true`, which are typically stored in a

`gradle.properties` file in a project root directory or `GRADLE_USER_HOME` environment

variable.

- Environment variables

For all available properties, see `./gradlew properties`

Gradle properties

```
// gradle.properties
kotlin.code.style=official
username=student

// build.gradle.kts
val username: String by project
val kotlinCodeStyle =
project.property("kotlin.code.style") as String
tasks.register("printProps") {
   doLast {
       println(username)
       println(kotlinCodeStyle)
       println(System.getProperty("idea.version"))
   }
}
```

Gradle properties

projectDir/module/build.gradle.kts

```
tasks.register("printProperty") {
    val prop: String? by project
    doLast {
        println(prop ?: "Not set")
    }
}
```

./gradlew :module:printProperty → Not set
If we add prop="Prop set" to projectDir/module/gradle.properties, then:
./gradlew :module:printProperty → Prop set
./gradlew -Pprop="Override prop":module:printProperty → Override prop

The Wrapper

The recommended way to execute any Gradle build is with the help of the Gradle Wrapper.

The Wrapper is a script that invokes a declared version of Gradle, downloading it beforehand if necessary.

You can build projects, run them, and more with the Wrapper locally without needing to have the global Gradle on your machine.

One way to upgrade the Gradle version is manually change the `distributionUrl` property in the Wrapper's `gradle-wrapper.properties` file.

– *Gradle Documentation*

Version catalog

You can configure all plugins, dependencies, and versions in one of two places:

- inside `settings.gradle.kts` via `dependencyResolutionManagement { versionCatalogs { … } }`
- In a special TOML file, conventionally named `libs.versions.toml` and stored in the Gradle folder.

The TOML file has a special structure:

```
[versions]
kotlin = "1.7.10"

[libraries]
junit-jupiter-api = {
     module = "org.junit.jupiter:junit-jupiter-api",
version.ref = "junit-jupiter"
}

[plugins]
kotlin-jvm = { id = "org.jetbrains.kotlin.jvm",
version.ref = "kotlin" }
```

Version catalog

```
// build.gradle.kts
plugins {
    val kotlinVersion =
libs.versions.kotlin.get()

id(libs.plugins.kotlin.jvm.get().pluginId)
version kotlinVersion
        apply false
}

allProjects {
    dependencies {

testImplementation(rootProject.libs.junit.jupi
    }
}
```

Here we've included one plugin and one dependency in the TOML file.

Summary

- Gradle is a build automation tool, which uses Kotlin DSL as the configuration language;
- Gradle is a JVM application, so it does not require any additional installation, and Gradle projects usually come with a gradle wrapper to build them;
- Gradle follows “Convention over Configuration ” principle;
- Configuration for the project and each sub-project is defined in `build.gradle.kts` files;
- Most of the Gradle functionality comes from tasks, which are defined in plugins, or implemented in the project itself;
- Gradle manages dependencies, as long as the user has defined all versions or conflict resolution strategy.

Gradle can do so much more!

Gradle support many additional features which we won't be covering today:

- Caching
- Multi-module projects
- More blocks:
  - `allprojects { }` and `subprojects { }`
  - `publishing { }`
  - `artifacts { }`
- Compatibility
- Resolution strategies
- Source sets

JETBRAINS | Kotlin

Thank you

